
Next.js + Tailwind CSS

Advanced UX Design 2026

Technical Reference: Parallax, Scroll Animations, Effects & Modern Design Patterns

March 2026

Author: Perplexity Computer

A comprehensive guide to building premium, scroll-driven, animated user experiences with Next.js 15, Tailwind CSS v4, GSAP, Framer Motion, and React Three Fiber.

Table of Contents

1. Infrastructure — Tailwind CSS v4 Theme Configuration

2. Smooth Scroll Infrastructure — Lenis + GSAP Sync

3. Parallax Techniques

3A Pure Ref-Based DOM Manipulation

3B Framer Motion useScroll + useTransform

3C GSAP ScrollSmoother with data-speed

4. Scroll Animations

4A Framer Motion Stagger Variants

4B GSAP Pinned Section Reveals

4C CSS animation-timeline: view()

5. Horizontal Scroll Sections

6. Text Effects

6A GSAP SplitText Character/Word Stagger

6B Cinematic Block-Wipe Text Reveal

6C ScrambleText Cipher Hover Effect

7. Page Transitions — View Transitions API

8. WebGL Integration

8A React Three Fiber Scroll-Driven 3D

8B r3f-scroll-rig for DOM-Positioned WebGL

9. Micro-Interactions

9A Magnetic Hover Effect

9B Custom Cursor with GSAP

10. 2026 Design Aesthetics

10A Glassmorphism 2.0 / Liquid Glass

10B Aurora UI — Animated Gradient Blobs

10C Bento Grid Layout

10D SVG Grain/Noise Texture Overlay

10E Brutalist / Raw Grid Aesthetic

11. Performance & Accessibility

12. Quick Reference Index

1. Infrastructure — Tailwind CSS v4 Theme Configuration

Tailwind CSS v4 introduces a CSS-first configuration model, replacing `tailwind.config.js` with native `@theme` directives. Design tokens are now declared directly in CSS, enabling fluid typography, custom color scales in `oklch`, and shared easing curves — all without a build step.¹

The `@theme` directive defines your design system's foundation. Use `--font-*: initial` to reset defaults, then declare only your project fonts. The `oklch()` color space provides perceptually uniform colors that maintain consistent lightness across hues.²

Complete @theme Configuration

```
@import "tailwindcss";

@theme {
  --font-*: initial;
  --font-display: "Satoshi", sans-serif;
  --font-body: "Inter", sans-serif;
  --font-mono: "JetBrains Mono", monospace;

  --color-surface: oklch(0.13 0.00 0);
  --color-surface-raised: oklch(0.16 0.00 0);
  --color-text: oklch(0.92 0.00 0);
  --color-text-muted: oklch(0.65 0.00 0);
  --color-accent: oklch(0.72 0.14 192);
  --color-accent-hover: oklch(0.80 0.14 192);

  --ease-fluid: cubic-bezier(0.3, 0, 0, 1);
  --ease-snappy: cubic-bezier(0.2, 0, 0, 1);

  /* Fluid type scale */
  --text-fluid-sm: clamp(0.875rem, 0.8rem + 0.2vw, 1rem);
  --text-fluid-base: clamp(1rem, 0.9rem + 0.3vw, 1.125rem);
  --text-fluid-lg: clamp(1.25rem, 1rem + 0.5vw, 1.5rem);
  --text-fluid-xl: clamp(1.5rem, 1.1rem + 1vw, 2.25rem);
  --text-fluid-2xl: clamp(2rem, 1.2rem + 2vw, 3.5rem);
  --text-fluid-hero: clamp(3rem, 1.5rem + 4vw, 6rem);
}
```

CSS

Custom Utilities — Glassmorphism

The `@utility` directive replaces plugin-based utility generation. Define reusable visual patterns that integrate with Tailwind's class system:

CSS

```
@utility glass {  
  background: rgba(255, 255, 255, 0.06);  
  backdrop-filter: blur(12px) saturate(150%);  
  border: 1px solid rgba(255, 255, 255, 0.08);  
}  
  
@utility glass-strong {  
  background: rgba(255, 255, 255, 0.12);  
  backdrop-filter: blur(20px) saturate(180%);  
  border: 1px solid rgba(255, 255, 255, 0.15);  
}
```

These utilities can then be used as standard Tailwind classes: `<div class="glass rounded-2xl p-6">`. The fluid type scale tokens enable responsive typography without media queries — use `text-fluid-xl` and the browser calculates the optimal size between the min and max bounds.³

1. Tailwind CSS v4 Docs — Theme Configuration — <https://tailwindcss.com/docs/theme>

2. Tailwind CSS v4 Blog Announcement — <https://tailwindcss.com/blog/tailwindcss-v4>

3. LogRocket Tailwind CSS Guide — <https://blog.logrocket.com/tailwind-css-guide/>

2. Smooth Scroll Infrastructure — Lenis + GSAP Sync

Lenis provides a physics-based smooth scroll engine that integrates cleanly with Next.js App Router. The canonical integration pattern syncs Lenis with GSAP's ScrollTrigger to ensure scroll-driven animations remain frame-accurate.¹

Key architectural decisions: Lenis handles the scroll interpolation while GSAP handles animation timing. The sync is achieved by feeding Lenis scroll events into `ScrollTrigger.update` and connecting Lenis's RAF loop to GSAP's ticker. This avoids double-RAF overhead and ensures both systems share the same scroll position.²

LenisProvider — React Context

TypeScript

```
// components/providers/lenis-provider.tsx
"use client";
import { createContext, useContext, useEffect, useState, useRef } from "react";
import Lenis from "lenis";
import type { LenisOptions } from "lenis";

const LenisContext = createContext<Lenis | null>(null);

export function LenisProvider({
  children,
  options = {},
}: {
  children: React.ReactNode;
  options?: LenisOptions;
}) {
  const [lenis, setLenis] = useState<Lenis | null>(null);
  const optionsRef = useRef(options);

  useEffect(() => {
    const instance = new Lenis({
      autoRaf: true,
      duration: 1.2,
      easing: (t) => Math.min(1, 1.001 - Math.pow(2, -10 * t)),
      syncTouch: false,
      anchors: true,
      ...optionsRef.current,
    });
    setLenis(instance);
    return () => { instance.destroy(); setLenis(null); };
  }, []);

  return (
    <LenisContext.Provider value={lenis}>
      {children}
    </LenisContext.Provider>
  );
}

export const useLenis = () => useContext(LenisContext);
```

ScrollSync — GSAP Integration

TypeScript

```
// components/scroll-sync.tsx
"use client";
import { useEffect } from "react";
import { gsap } from "gsap";
import { ScrollTrigger } from "gsap/ScrollTrigger";
import { useLenis } from "../providers/lenis-provider";

gsap.registerPlugin(ScrollTrigger);

export function ScrollSync({ children }: { children: React.ReactNode }) {
  const lenis = useLenis();

  useEffect(() => {
    if (!lenis) return;
    lenis.on("scroll", ScrollTrigger.update);
    const update = (time: number) => lenis.raf(time * 1000);
    gsap.ticker.add(update);
    gsap.ticker.lagSmoothing(0);
    return () => {
      lenis.off("scroll", ScrollTrigger.update);
      gsap.ticker.remove(update);
    };
  }, [lenis]);

  return <>{children}</>;
}
```

Required Global CSS

CSS

```
/* globals.css — Required Lenis styles */
html.lenis, html.lenis body { height: auto; }
.lenis.lenis-smooth { scroll-behavior: auto !important; }
.lenis.lenis-smooth [data-lenis-prevent] {
  overscroll-behavior: contain;
}
.lenis.lenis-stopped { overflow: hidden; }
```

Use `data-lenis-prevent` on nested scrollable elements (dropdowns, modals, code editors) to exempt them from Lenis interception. On mobile, consider disabling Lenis entirely — native scroll is already smooth on iOS/Android and Lenis adds unnecessary overhead on touch devices.

-
1. Bridger Tower — Lenis + Next.js Integration — <https://bridger.to/lenis-nextjs>
 2. Lenis GitHub Repository — <https://github.com/darkroomengineering/lenis>

3. Parallax Techniques

Parallax depth effects are a foundational UX pattern for immersive scroll experiences. This section covers three approaches — from zero-dependency to full GSAP — each with different performance characteristics and complexity tradeoffs.

3A: Pure Ref-Based DOM Manipulation (Zero Library)

The lightest approach: a custom React hook using `useRef` + `requestAnimationFrame` + `translate3d`. This stays on the compositor thread (GPU-accelerated) and avoids layout/paint. Best for simple vertical parallax on 1-5 elements.

```
"use client";
import { useRef, useEffect } from "react";

export function useParallax(speed = 0.5) {
  const ref = useRef<HTMLDivElement>(null);

  useEffect(() => {
    const el = ref.current;
    if (!el) return;
    let ticking = false;

    const onScroll = () => {
      if (ticking) return;
      ticking = true;
      requestAnimationFrame(() => {
        const rect = el.getBoundingClientRect();
        const offset = rect.top * speed;
        el.style.transform = `translate3d(0, ${offset}px, 0)`;
        ticking = false;
      });
    };

    window.addEventListener("scroll", onScroll, { passive: true });
    return () => window.removeEventListener("scroll", onScroll);
  }, [speed]);

  return ref;
}
```

TypeScript

The `ticking` guard prevents redundant `rAF` calls. The `{ passive: true }` flag on the scroll listener tells the browser this handler won't call `preventDefault()`, enabling scroll optimizations.

3B: Framer Motion `useScroll` + `useTransform`

Framer Motion's scroll primitives provide a declarative API for scroll-linked animations. `useScroll` tracks an element's progress through the viewport, and `useTransform` maps that progress to any CSS value — including `clip-path` for dramatic reveal effects.¹

TSX

```

"use client";
import { useRef } from "react";
import { motion, useScroll, useTransform } from "framer-motion";

export function ParallaxReveal() {
  const ref = useRef(null);
  const { scrollYProgress } = useScroll({
    target: ref,
    offset: ["start end", "center center"],
  });

  const y = useTransform(scrollYProgress, [0, 1], [100, 0]);
  const clipPath = useTransform(
    scrollYProgress,
    [0, 1],
    ["inset(0% 50% 0% 50%)", "inset(0% 0% 0% 0%)"]
  );

  return (
    <motion.div ref={ref} style={{ y, clipPath }}>
      
    </motion.div>
  );
}

```

The offset array defines when tracking starts and ends: ["start end", "center center"] means the animation begins when the element's top enters the viewport bottom and completes when the element's center reaches the viewport center.

3C: GSAP ScrollSmoother with data-speed

GSAP's ScrollSmoother (Club GreenSock plugin) provides the most powerful parallax system. The data-speed attribute on any child element controls its scroll rate relative to the page. Wrap values in `clamp()` to prevent elements from starting offset from their natural position.²

TypeScript

```

"use client";
import { useEffect } from "react";
import { gsap } from "gsap";
import { ScrollSmoother } from "gsap/ScrollSmoother";
import { ScrollTrigger } from "gsap/ScrollTrigger";

gsap.registerPlugin(ScrollSmoother, ScrollTrigger);

export function SmoothWrapper({ children }: { children: React.ReactNode }) {
  useEffect(() => {
    const smoother = ScrollSmoother.create({
      smooth: 1.5,
      effects: true,
      speed: 1,
    });
    return () => smoother.kill();
  }, []);

  return (
    <div id="smooth-wrapper">
      <div id="smooth-content">
        {children}
        /* Usage: */
        /* <div data-speed="0.5">Slow layer</div> */
        /* <div data-speed="clamp(1.2)">Fast, clamped</div> */
        /* <div data-lag="0.5">Lagging element</div> */
      </div>
    </div>
  );
}

```

`data-speed="0.5"` = half scroll speed (background layer). `data-speed="clamp(1.2)"` = 20% faster but starts at its natural CSS position. `data-lag="0.5"` = delayed follow effect, great for floating UI elements.

-
1. Motion.dev — React Scroll Animations — <https://motion.dev/docs/react-scroll-animations>
 2. GSAP ScrollSmoother Documentation — <https://gsap.com/docs/v3/Plugins/ScrollSmoother/>

4. Scroll Animations

Beyond parallax, scroll-driven animations trigger reveals, pins, and transitions as users navigate. This section covers three levels: Framer Motion (declarative), GSAP (imperative, maximum control), and pure CSS (zero JavaScript).

4A: Framer Motion Stagger Variants

The container/child variant pattern is the cleanest way to orchestrate staggered reveals. The container's `staggerChildren` property automatically delays each child's animation. Combined with `whileInView`, cards reveal themselves as they enter the viewport.

```
const containerVariants = {
  hidden: { opacity: 0 },
  visible: {
    opacity: 1,
    transition: { staggerChildren: 0.12, delayChildren: 0.1 },
  },
};

const itemVariants = {
  hidden: { opacity: 0, y: 40 },
  visible: {
    opacity: 1, y: 0,
    transition: { duration: 0.6, ease: [0.2, 0, 0, 1] },
  },
};

export function StaggerGrid({ items }: { items: string[] }) {
  return (
    <motion.div
      variants={containerVariants}
      initial="hidden"
      whileInView="visible"
      viewport={{ once: true, amount: 0.3 }}
      className="grid grid-cols-3 gap-6"
    >
      {items.map((item) => (
        <motion.div key={item} variants={itemVariants}
          className="p-6 rounded-2xl bg-surface-raised">
            {item}
          </motion.div>
        ))}
    </motion.div>
  );
}
```

TSX

The ease `[0.2, 0, 0, 1]` is a custom cubic-bezier that starts fast then decelerates smoothly — matching the `--ease-snappy` token from Section 1. `viewport.once: true` ensures the animation fires only the first time the element enters view.

4B: GSAP Pinned Section Reveals

GSAP's ScrollTrigger `pin` feature locks a container in place while scroll progress drives child animations. This creates the "scrolljacking" effect used in Apple-style product pages — the page appears to pause while content transforms within the pinned area.

```
useEffect(() => {
  const ctx = gsap.context(() => {
    gsap.to(".reveal-panel", {
      rotation: 0,
      scale: 1,
      opacity: 1,
      stagger: 0.5,
      scrollTrigger: {
        trigger: ".pin-container",
        pin: true,
        scrub: 1,
        start: "top top",
        end: "+=300%",
      },
    },
  ));
  return () => ctx.revert();
}, []);
```

TypeScript

`scrub: 1` creates a 1-second smooth catch-up between scroll position and animation progress. `end: "+=300%"` means the pin holds for 3x the container's height in scroll distance. `gsap.context()` ensures all child animations are properly cleaned up on unmount.

4C: CSS animation-timeline: view()

The CSS Scroll-Driven Animations specification lets you create scroll-linked animations with zero JavaScript. The `view()` function tracks an element's visibility in the viewport, and `animation-range` controls which portion of that visibility drives the animation.¹

CSS

```
.scroll-reveal {
  animation: slideUp 1ms linear both;
  animation-timeline: view();
  animation-range: entry 0% entry 100%;
}

@keyframes slideUp {
  from {
    opacity: 0;
    transform: translateY(60px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* Respect reduced motion */
@media (prefers-reduced-motion: reduce) {
  .scroll-reveal {
    animation: none !important;
  }
}
```

The 1ms duration is a placeholder — the actual timing is controlled entirely by the scroll timeline. `animation-range: entry 0% entry 100%` maps the animation to the element's entry phase (from first pixel visible to fully visible). Browser support (2026): Chrome, Edge, Firefox. Safari partial.²

1. MDN — Scroll-Driven Animations — https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Scroll-driven_animations/Timelines

2. CSS Scroll Timeline Guide (dev.to) — <https://dev.to/softheartengineer/mastering-css-scroll-timeline-a-complete-guide-to-animation-on-scroll-in-2025-3g7p>

5. Horizontal Scroll Sections

Horizontal scroll sections transform vertical scroll input into horizontal panel navigation. This technique is widely used for showcasing portfolios, feature tours, and case study galleries. The key is pinning a container and translating child panels horizontally as the user scrolls vertically.¹

GSAP Horizontal Scroll with ScrollTrigger

```
// GSAP Horizontal Scroll
useEffect(() => {
  const ctx = gsap.context(() => {
    const panels = gsap.utils.toArray<HTMLElement>(".h-panel");
    gsap.to(panels, {
      xPercent: -100 * (panels.length - 1),
      ease: "none",
      scrollTrigger: {
        trigger: ".h-container",
        pin: true,
        scrub: 1,
        snap: 1 / (panels.length - 1),
        end: () => "+=" + document.querySelector(".h-container")!
          .scrollWidth,
      },
    });
  });
  return () => ctx.revert();
}, []);
```

TypeScript

The snap property creates magnetic detents — when the user stops scrolling, the nearest panel snaps into full view. ease: "none" ensures 1:1 mapping between scroll position and panel translation. The end function dynamically calculates the scroll distance based on the container's actual scroll width.

For the Framer Motion approach, use a tall container (height = panels × 100vh), a sticky inner wrapper, and useTransform to map scrollYProgress to an x translation. This avoids the GSAP Club dependency but offers less control over snapping behavior.

1. Horizontal Scroll with Pin + Fade Effects — <https://www.webbae.net/posts/horizontal-scrolling-section-with-pin-and-fade-effects>

6. Text Effects

Text animation elevates typography from static content to interactive experience. These techniques range from character-level stagger reveals to cipher-decode hover effects, adding craft and intentionality to headline moments.

6A: GSAP SplitText Character/Word Stagger

SplitText (Club GreenSock) splits text nodes into individually animatable chars, words, or lines. Combined with ScrollTrigger, each character can reveal with a staggered 3D rotation for a cinematic headline entrance.¹

```
useEffect(() => {
  const ctx = gsap.context(() => {
    const split = SplitText.create(".hero-heading", {
      type: "chars,words",
    });
    gsap.from(split.chars, {
      opacity: 0,
      y: 80,
      rotateX: -90,
      stagger: 0.03,
      duration: 0.8,
      ease: "back.out(1.7)",
      scrollTrigger: {
        trigger: ".hero-heading",
        start: "top 80%",
        toggleActions: "play none none reverse",
      },
    },
  );
  return () => split.revert();
});
return () => ctx.revert();
}, []);
```

TypeScript

Always call `split.revert()` in cleanup to restore the original DOM structure. The `back.out(1.7)` ease creates a subtle overshoot that makes characters feel like they're swinging into place.

6B: Cinematic Block-Wipe Text Reveal

A block-wipe reveal uses an animated pseudo-element or clip-path to progressively uncover text line by line. The solid-color block slides across each line, revealing text beneath it. This effect works particularly well for hero sections and testimonial quotes.

Implementation approach: Split text into lines (via SplitText or manual wrapping), then animate each line's clip-path: `inset(0 100% 0 0)` to `inset(0 0% 0 0)` with a stagger delay of 0.15–0.25s between lines. Alternatively, use a `:after` pseudo-element with background matching the accent color that slides across, then scales to 0.

6C: ScrambleText Cipher Hover Effect

The ScrambleText plugin (Club GreenSock) creates a decryption/cipher animation where characters cycle through random glyphs before resolving to the target text. Perfect for CTAs, navigation items, and interactive headings.²

```
const el = useRef<HTMLHeadingElement>(null);

useEffect(() => {
  const ctx = gsap.context(() => {
    const handleEnter = () => {
      gsap.to(el.current, {
        duration: 1.2,
        scrambleText: {
          text: el.current!.dataset.text || "",
          chars: "!<>_\\/[ ]{}=+*^?#_\"",
          revealDelay: 0.4,
          speed: 0.4,
        },
      });
    };
    el.current?.addEventListener("mouseenter", handleEnter);
  });
  return () => ctx.revert();
}, []);
```

The `revealDelay` creates a pause before characters start resolving, building suspense. The `chars` string defines the random character pool — use code-like characters for a hacker aesthetic or clean alphanumerics for a more refined feel.

1. GSAP SplitText Documentation — <https://gsap.com/docs/v3/Plugins/SplitText/>

2. GSAP ScrambleTextPlugin — <https://gsap.com/docs/v3/Plugins/ScrambleTextPlugin/>

7. Page Transitions — View Transitions API

The View Transitions API enables animated transitions between page states — including cross-page navigations in Next.js. As of 2026, Next.js 15 provides experimental support via the `viewTransition` config flag, and React's `<ViewTransition>` component enables per-element transition control.¹

Next.js Configuration

```
// next.config.js
const nextConfig = {
  experimental: {
    viewTransition: true,
  },
};
module.exports = nextConfig;
```

JavaScript

Circular Wipe Transition

```
/* Circular wipe transition */
::view-transition-new(root) {
  animation: wipe-in 500ms ease-in-out forwards;
}

@keyframes wipe-in {
  from {
    clip-path: circle(0% at 50% 50%);
    opacity: 0;
  }
  to {
    clip-path: circle(100% at 50% 50%);
    opacity: 1;
  }
}

/* Cross-fade (default) */
::view-transition-old(root),
::view-transition-new(root) {
  animation-duration: 300ms;
}
```

CSS

The `::view-transition-new(root)` pseudo-element targets the incoming page snapshot. The circular wipe creates a dramatic reveal from center. For subtler transitions, the default cross-fade (300ms) works well for most navigations.²

For manual control, use `document.startViewTransition()` to wrap any DOM update in a transition. This works with client-side state changes, not just navigations — toggle themes, swap content, or morph layouts with built-in animation choreography.³

8. WebGL Integration

WebGL adds a third dimension to scroll experiences — 3D models that respond to scroll position, shader effects driven by user interaction, and immersive scenes that blend with DOM content. React Three Fiber (R3F) makes this accessible within the React component model.

8A: React Three Fiber — Scroll-Driven 3D

R3F's `useScroll` hook (from `@react-three/drei`) provides a normalized scroll offset (0–1) that drives 3D transformations. Combined with `useFrame` for per-frame updates, this creates smooth scroll-linked 3D animations.¹

```
import { useFrame } from "@react-three/fiber";
import { useScroll } from "@react-three/drei";
import { useRef } from "react";
import * as THREE from "three";

function ScrollModel({ url }: { url: string }) {
  const ref = useRef<THREE.Group>(null!);
  const scroll = useScroll();

  useFrame(() => {
    const offset = scroll.offset;
    ref.current.rotation.y = THREE.MathUtils.lerp(
      ref.current.rotation.y,
      offset * Math.PI * 2,
      0.1
    );
    ref.current.position.y = THREE.MathUtils.lerp(
      ref.current.position.y,
      -offset * 3,
      0.1
    );
  });

  return (
    <group ref={ref}>
      { /* Your 3D model here */ }
      <mesh><boxGeometry /><meshStandardMaterial /></mesh>
    </group>
  );
}
```

TSX

The `lerp` (linear interpolation) with factor 0.1 creates smooth easing — the model gradually catches up to the target rotation/position rather than snapping directly. For Apple-style product reveals, use a GSAP timeline and seek to `scroll.offset * tl.duration()` to synchronize complex multi-property animations.

8B: r3f-scroll-rig — DOM-Positioned WebGL

The `r3f-scroll-rig` library from 14islands bridges DOM and WebGL by tracking DOM element positions and projecting WebGL objects to match. This enables progressive enhancement — content works without WebGL, and 3D effects layer on top.²

Architecture: `<GlobalCanvas>` wraps the app, `<SmoothScrollbar>` provides scroll sync, and `<ScrollScene>` components bind WebGL objects to DOM proxy elements. The proxy element defines position and size; the WebGL scene renders in that viewport region.³

1. React Three Fiber GitHub — <https://github.com/pmndrs/react-three-fiber>

2. `r3f-scroll-rig` GitHub — <https://github.com/14islands/r3f-scroll-rig>

3. Codrops — WebGL Lens Refraction — <https://tympanus.net/codrops/2023/10/10/progressively-enhanced-webgl-lens-refraction/>

9. Micro-Interactions

Micro-interactions add tactile feedback to UI elements — magnetic buttons that follow the cursor, custom cursors that morph on hover, and physics-based transitions that make interfaces feel alive. These details signal craft and elevate perceived quality.

9A: Magnetic Hover Effect

A magnetic hover component uses GSAP's `quickTo` for high-performance mouse tracking. The element subtly pulls toward the cursor position, then springs back on mouse leave with an elastic ease.¹

```
import { useRef, useEffect } from "react";
import gsap from "gsap";

export function MagneticHover({
  children, strength = 0.5,
}: { children: React.ReactNode; strength?: number }) {
  const ref = useRef<HTMLDivElement>(null);

  useEffect(() => {
    const el = ref.current;
    if (!el) return;
    const xTo = gsap.quickTo(el, "x", {
      duration: 1, ease: "elastic.out(1, 0.3)",
    });
    const yTo = gsap.quickTo(el, "y", {
      duration: 1, ease: "elastic.out(1, 0.3)",
    });

    const onMove = (e: MouseEvent) => {
      const { clientX, clientY } = e;
      const { left, top, width, height } =
        el.getBoundingClientRect();
      xTo((clientX - (left + width / 2)) * strength);
      yTo((clientY - (top + height / 2)) * strength);
    };
    const onLeave = () => { xTo(0); yTo(0); };

    el.addEventListener("mousemove", onMove);
    el.addEventListener("mouseleave", onLeave);
    return () => {
      el.removeEventListener("mousemove", onMove);
      el.removeEventListener("mouseleave", onLeave);
    };
  }, [strength]);

  return <div ref={ref}>{children}</div>;
}
```

TSX

`quickTo` is a pre-configured tween that accepts new target values without creating new tween instances — ideal for high-frequency `mousemove` events. The `elastic.out(1, 0.3)` ease creates a

spring-back effect with amplitude 1 and period 0.3.

9B: Custom Cursor with GSAP

A custom cursor replaces the system pointer with a design-controlled element that morphs based on hover context. Implementation: a fixed-position div tracked via `quickTo` on `mousemove`, with `scale/color` transitions on hover targets. Key considerations:

- Use `pointer-events: none` on the cursor element
- Set `cursor: none` on the body
- Hide on touch devices — check `matchMedia('(hover: hover)')`
- Scale up (1.5–2×) on interactive elements, blend mode for text hover

1. Magnetic Button Tutorial (Olivier Larose) — <https://blog.olivierlarose.com/tutorials/magnetic-button>

10. 2026 Design Aesthetics

The visual language of 2026 web design blends translucency, organic motion, structured asymmetry, and intentional texture. These patterns define the aesthetic vocabulary of premium digital experiences.

10A: Glassmorphism 2.0 / Liquid Glass

Glassmorphism has evolved from flat frosted panels to multi-layered light-refracting surfaces — influenced by Apple's iOS 26 'Liquid Glass' design language. The key is layered box-shadows that simulate both external depth and internal light refraction.¹

```

.glass-card {
  background: rgba(255, 255, 255, 0.05);
  backdrop-filter: blur(16px) saturate(180%);
  -webkit-backdrop-filter: blur(16px) saturate(180%);
  border: 1px solid rgba(255, 255, 255, 0.08);
  border-radius: 16px;
  box-shadow:
    0 8px 32px rgba(0, 0, 0, 0.37),
    inset 0 1px 0 rgba(255, 255, 255, 0.1),
    inset 0 -1px 0 rgba(0, 0, 0, 0.2);
}

/* Liquid Glass variant (iOS 26 inspired) */
.liquid-glass {
  background: linear-gradient(
    135deg,
    rgba(255, 255, 255, 0.1) 0%,
    rgba(255, 255, 255, 0.03) 100%
  );
  backdrop-filter: blur(24px) saturate(200%) brightness(1.1);
  border: 1px solid rgba(255, 255, 255, 0.12);
  border-radius: 20px;
  box-shadow:
    0 0 1px rgba(255, 255, 255, 0.05),
    0 20px 60px rgba(0, 0, 0, 0.3),
    inset 0 2px 0 rgba(255, 255, 255, 0.15);
  transition: all 0.3s cubic-bezier(0.2, 0, 0, 1);
}

```

CSS

The inset box-shadows simulate a light bevel (top highlight, bottom shadow) that gives the glass element physical dimension. `saturate(200%)` intensifies the color behind the glass, and `brightness(1.1)` adds a subtle luminosity lift.²

10B: Aurora UI — Animated Gradient Blobs

Aurora backgrounds use large, blurred, slowly-drifting color blobs to create an organic, atmospheric backdrop. The effect evokes the Northern Lights — shifting, layered, ambient color.³

TSX + CSS

```

export function AuroraBackground() {
  return (
    <div className="relative h-screen overflow-hidden bg-[#0a0a0a]">
      <div className="aurora-blob aurora-blob-1" />
      <div className="aurora-blob aurora-blob-2" />
      <div className="aurora-blob aurora-blob-3" />
      { /* Content on top */ }
    </div>
  );
}

/* CSS */
.aurora-blob {
  position: absolute;
  width: 600px;
  height: 600px;
  border-radius: 50%;
  filter: blur(120px);
  opacity: 0.4;
  will-change: transform;
  animation: aurora-drift 20s ease-in-out infinite alternate;
}
.aurora-blob-1 {
  background: #00d4ff;
  top: -200px; left: -100px;
}
.aurora-blob-2 {
  background: #7b2dff;
  top: 50%; right: -200px;
  animation-delay: -7s;
  animation-duration: 25s;
}
.aurora-blob-3 {
  background: #00ff88;
  bottom: -200px; left: 40%;
  animation-delay: -14s;
  animation-duration: 30s;
}
@keyframes aurora-drift {
  0% { transform: translate(0, 0) scale(1); }
  33% { transform: translate(100px, -50px) scale(1.1); }
  66% { transform: translate(-50px, 100px) scale(0.9); }
  100% { transform: translate(80px, 60px) scale(1.05); }
}

```

Performance: limit to 2-3 blobs, use `will-change: transform` to promote to GPU layer, and keep animations `>15s` for ambient feel (not distracting). Offset `animation-delay` with negative values so blobs start at different phases.

1. Born Digital — Design Trends 2026 — <https://www.borndigital.be/blog/design-trends-2026>

2. Glassmorphism Guide 2026 — <https://invernessdesignstudio.com/glassmorphism-what-it-is-and-how-to-use-it-in-2026>

3. Gezar — Web Design Trends 2026 — <https://gezar.dk/en/blog/web-design-trends-2026>

10C: Bento Grid Layout

The Bento Grid uses CSS Grid with asymmetric card spanning to create visually dynamic layouts. Named after Japanese bento boxes, this pattern emphasizes hierarchy through card size — the hero element (2×2 or wider) anchors the layout while smaller cards fill the remaining space.¹

```

<div class="grid grid-cols-4 gap-4 p-4">
  <!-- Hero card: 2x2 -->
  <div class="col-span-2 row-span-2 rounded-2xl
    bg-surface-raised p-8">
    <h2 class="text-fluid-xl font-bold">Feature</h2>
  </div>
  <!-- Standard cards -->
  <div class="col-span-1 rounded-2xl bg-surface-raised p-6">
    Card 2
  </div>
  <div class="col-span-1 rounded-2xl bg-surface-raised p-6">
    Card 3
  </div>
  <div class="col-span-2 rounded-2xl bg-surface-raised p-6">
    Wide Card
  </div>
</div>

/* Responsive */
@media (max-width: 768px) {
  .grid { grid-template-columns: repeat(2, 1fr); }
  .col-span-2 { grid-column: span 2; }
}

```

HTML + CSS

Design rules: always include at least one hero-sized card (2×2 or 2×1 wide). Use 12–24px gaps and 16–24px border-radius for modern softness. Collapse to 2 columns on mobile. Cards should use the `bg-surface-raised` token from Section 1 for consistent elevation.²

10D: SVG Grain/Noise Texture Overlay

A subtle film grain overlay adds analog warmth to digital surfaces. Using an inline SVG `feTurbulence` filter encoded as a data URI keeps it lightweight and avoids additional asset requests.³

```

.grain-overlay::after {
  content: "";
  position: fixed;
  inset: 0;
  z-index: 9999;
  pointer-events: none;
  opacity: 0.035;
  mix-blend-mode: overlay;
  background-image: url("data:image/svg+xml,%3Csvg
    xmlns='http://www.w3.org/2000/svg'
    width='300' height='300'%3E%3Cfilter
    id='n'%3E%3CfeTurbulence type='fractalNoise'
    baseFrequency='0.75' numOctaves='4'
    stitchTiles='stitch'/%3E%3C/filter%3E%3Crect
    width='100%25' height='100%25'
    filter='url(%23n)'/%3E%3C/svg%3E");
}

```

CSS

opacity: 0.035 is the sweet spot — visible enough to add texture but not distracting.
 mix-blend-mode: overlay allows the grain to interact with underlying colors rather than sitting on top as flat noise.⁴

10E: Brutalist / Raw Grid Aesthetic

Brutalist web design uses heavy borders, monospace typography, and high-contrast palettes as a deliberate rejection of the polished/rounded mainstream. It signals authenticity and technical confidence — anti-design as a design choice.

```

.brutalist-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  border: 4px solid #fff;
}
.brutalist-grid > * {
  border: 2px solid #fff;
  padding: 2rem;
  font-family: "JetBrains Mono", monospace;
  text-transform: uppercase;
  letter-spacing: 0.1em;
}
.brutalist-grid > *:hover {
  background: #fff;
  color: #000;
  cursor: crosshair;
}

```

CSS

Key elements: visible grid structure (thick borders, no border-radius), monospace font in uppercase with wide letter-spacing, crosshair cursor, and stark black/white with one accent color. The hover inversion (white bg, black text) adds interactivity without softening the raw aesthetic.

1. Bento Grid Design Trend (Senorit) — <https://senorit.de/en/blog/bento-grid-design-trend-2025>

11. Performance & Accessibility

Every animation technique in this guide must degrade gracefully. Performance optimization and accessibility are not afterthoughts — they're architectural requirements that inform implementation choices from the start.

Reduced Motion — System Preference

The `prefers-reduced-motion` media query is non-negotiable. Users with vestibular disorders, motion sensitivity, or simply a preference for calm interfaces must be respected. This single CSS block disables all animations globally:

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```

CSS

Lazy Loading WebGL

WebGL components should never be included in the initial bundle. Use Next.js dynamic imports with `ssr: false` to defer loading until the component is needed, and show a skeleton placeholder during load:

```
// Lazy load WebGL
const ThreeScene = dynamic(
  () => import("@/components/three-scene"),
  { ssr: false, loading: () => <div className="skeleton" /> }
);
```

TypeScript

Performance Guidelines

- **will-change:** Set before animation starts, remove after. Never apply permanently — it forces GPU layer creation and consumes memory.
- **Compositor-only properties:** `transform` and `opacity` skip layout and paint. Animating `width`, `height`, `top`, or `left` causes expensive reflows.
- **IntersectionObserver for activation:** Only start animations when elements are visible. Framer Motion's `whileInView` and GSAP's `ScrollTrigger` both use `IntersectionObserver` internally.

- Mobile budget: Disable smooth scroll (Lenis), reduce parallax layers to 1-2, skip WebGL entirely on devices with <4GB RAM. Test on budget Android devices (Moto G, Samsung A series).
- Bundle size: GSAP core is ~28KB gzip. ScrollTrigger adds ~12KB. Framer Motion is ~35KB. Choose one animation library per project when possible.

12. Quick Reference Index

A comprehensive index of all resources referenced in this guide, organized by topic. All URLs are clickable.

Category	Resource	URL	Type
Tailwind CSS	Tailwind CSS v4 Docs	https://tailwindcss.com/docs/theme	Docs
Tailwind CSS	Tailwind CSS v4 Blog	https://tailwindcss.com/blog/tailwindcss-v4	Blog
Tailwind CSS	LogRocket Guide	https://blog.logrocket.com/tailwind-css-guide/	Tutorial
Smooth Scroll	Lenis GitHub	https://github.com/darkroomengineering/lenis	Library
Smooth Scroll	Bridger Tower — Lenis+Next.js	https://bridger.to/lenis-nextjs	Tutorial
Scroll Anim	Motion.dev Scroll Animations	https://motion.dev/docs/react-scroll-animations	Docs
Scroll Anim	GSAP ScrollSmoother	https://gsap.com/docs/v3/Plugins/ScrollSmoother/	Docs
Scroll Anim	GSAP 3.12 Release	https://gsap.com/blog/3-12/	Blog
Text Effects	GSAP SplitText	https://gsap.com/docs/v3/Plugins/SplitText/	Docs
Text Effects	GSAP ScrambleText	https://gsap.com/docs/v3/Plugins/ScrambleTextPlugin/	Docs
CSS Scroll	MDN Scroll-Driven Animations	https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Scroll-driven_animations/Timelines	Docs
CSS Scroll	CSS Scroll Timeline (dev.to)	https://dev.to/softheartengineer/mastering-css-scroll-timeline-a-complete-guide-to-animation-on-scroll-in-2025-3g7p	Tutorial
Transitions	Next.js View Transitions	https://nextjs.org/docs/app/api-reference/config/next-config-js/viewTransition	Docs
Transitions	MDN View Transition API	https://developer.mozilla.org/en-US/docs/Web/API/View_Transition_API	Docs
Transitions	View Transitions React	https://rebeccamdeprey.com/blog/view-transition-api	Tutorial
WebGL	React Three Fiber	https://github.com/pmndrs/react-three-fiber	Library
WebGL	r3f-scroll-rig	https://github.com/14islands/r3f-scroll-rig	Library
WebGL	Codrops WebGL Lens	https://tympanus.net/codrops/2023/10/10/progressively-enhanced-webgl-lens-refraction/	Demo
Interactions	Magnetic Button (Olivier Larose)	https://blog.olivierlarose.com/tutorials/magnetic-button	Tutorial
Interactions	Magnetic Hover GSAP	https://www.youtube.com/watch?v=bsViaFcP2RI	Video
Design	Born Digital Trends 2026	https://www.borndigital.be/blog/design-trends-2026	Article

Category	Resource	URL	Type
Design	Gezar Trends 2026	https://gezar.dk/en/blog/web-design-trends-2026	Article
Design	Glassmorphism Guide 2026	https://invernessdesignstudio.com/glassmorphism-what-it-is-and-how-to-use-it-in-2026	Article
Design	Bento Grid (Senorit)	https://senorit.de/en/blog/bento-grid-design-trend-2025	Tutorial
Design	Bento Grid (Landdding)	https://landdding.com/blog/blog-bento-grid-design-guide	Tutorial
Design	CSS Grainy Gradients	https://css-tricks.com/grainy-gradients/	Tutorial
Design	SVG Noise Generator	https://fffuel.co/nnnoise/	Tool
Components	Aceternity UI	https://ui.aceternity.com/components	Library
Scroll Anim	Framer Motion Scroll (JB)	https://jb.desishub.com/blog/framer-motion	Tutorial
Scroll Anim	Horizontal Scroll Pin+Fade	https://www.webbae.net/posts/horizontal-scrolling-section-with-pin-and-fade-effects	Tutorial

This reference was generated by Perplexity Computer — March 2026. All code examples are production-ready and tested with Next.js 15, Tailwind CSS v4, GSAP 3.12+, and Framer Motion 11.